

Projeto QA - API de Tarefas

- Testes de API com Postman
- Projeto desenvolvido com Python (Flask)
- Foco em validação funcional e qualidade

Objetivo

- Garantir funcionamento correto dos endpoints
- Validar respostas e comportamento da API
- Simular cenários reais de uso

Stack e Ferramentas

- Postman (execução e organização de testes)
- Python (Flask) – API
- JSON – estrutura de dados
- HTTP/REST – comunicação

Endpoints Testados

- GET /tarefas → listar tarefas
- POST /tarefas → criar nova tarefa
- PUT /tarefas/{id} → atualizar tarefa
- DELETE /tarefas/{id} → remover tarefa

Validações Implementadas

- Status codes (200, 201, 404, etc.)
- Estrutura e conteúdo do JSON
- Persistência dos dados após operações
- Mensagens de erro consistentes

Cenários de Teste

- Fluxo positivo (criação e consulta)
- Atualização de dados existentes
- Remoção de registros
- Cenários negativos (ID inexistente, dados inválidos)

Desafios Encontrados

- Entendimento da lógica da API
- Organização eficiente dos testes
- Validação detalhada das respostas
- Simulação de cenários negativos

Como Superei

- Divisão dos testes por endpoint
- Uso de coleções no Postman
- Testes incrementais (step-by-step)
- Revisão de respostas e ajustes contínuos

Boas Práticas de QA Aplicadas

- Testes baseados em requisitos
- Cobertura de cenários positivos e negativos
- Clareza na validação de resultados
- Organização e reutilização de testes

Diferenciais do Projeto

- Simulação real de testes de API
- Uso prático de ferramentas do mercado
- Estrutura pronta para automação futura
- Base para integração com CI/CD

Próximos Passos

- Automatizar testes com Python (requests/pytest)
- Integrar com CI/CD
- Aumentar cobertura de testes
- Adicionar testes de performance

Conclusão

- Consolidação de conhecimentos em testes de API
- Experiência prática com Postman
- Evolução para testes mais avançados